Historical project report: scores and figures not independently reproduced. See RESULTS.md.

Handwritten Text Recognition

Irisbaev Nuriddin

Sapienza University of Rome - AI LAB

Abstract

This paper presents a Handwritten Text Recognition (HTR) system that converts images of handwritten words into machine-readable text. We adopt the classical *Convolutional Recurrent Neural Network* (CRNN) architecture: a convolutional encoder extracts a sequence of visual features, a two-layer bidirectional Long Short-Term Memory (BiLSTM) network models the left-to-right and right-to-left context, and a linear projection produces per-time-step character probabilities. The model is trained with the *Connectionist Temporal Classification* (CTC) loss, which removes the need for character-level alignment and naturally handles labels of variable length. Predictions are obtained through greedy CTC decoding. Evaluated on the word-level IAM Handwriting Database, the model converges to a **Character Error Rate (CER) of** 0.012 and a **Word Error Rate (WER) of** 0.027 on the validation set, and recognises 89.5% of a held-out test subset exactly. Error analysis shows that accuracy is strongly correlated with word length and with the model's own confidence, confirming that the system's uncertainty is well aligned with its mistakes.

1 Introduction

Handwriting remains one of the most natural ways for humans to record information, yet it is difficult for machines to read: the same character varies enormously across writers, pens, and styles, and individual letters are not cleanly separated in cursive script. *Handwritten Text Recognition* (HTR) is the task of transforming an image of handwritten text into the corresponding sequence of printed characters. It underpins applications such as historical-document digitisation, automatic form processing, and note transcription.

The central difficulty of HTR is *alignment*: we know the target text of a word image, but not which pixels correspond to which character. Segmentation-based methods that first cut a word into individual letters are brittle, because reliable character segmentation is itself almost as hard as recognition. Segmentation-free sequence models avoid this problem entirely. In this work we build such a model around the CRNN architecture and the CTC loss, which together allow end-to-end training using only (image, transcription) pairs.

Our contributions are: (i) a compact, fully reproducible CRNN pipeline for word-level HTR; (ii) an end-to-end training procedure with CTC loss and greedy decoding; and (iii) a detailed quantitative and qualitative error analysis on the IAM benchmark.

2 Related Work

Early HTR systems relied on Hidden Markov Models with hand-crafted features. The introduction of CTC [1] enabled recurrent networks to be trained directly on unsegmented sequences. Shi et al. [2] combined a convolutional encoder with a recurrent sequence model and CTC into the CRNN, which became the standard baseline for scene-text and handwriting recognition. More recent approaches replace the recurrent stage with attention or Transformer encoders (e.g. TrOCR [4]), trading higher accuracy for larger model size and data requirements. We adopt the CRNN as a strong, lightweight baseline that is well suited to a single-GPU training budget.

3 Dataset

We use the word-level partition of the **IAM Handwriting Database**, the most widely used English offline-handwriting benchmark. Each sample is a grayscale image of a single handwritten word together with its transcription, parsed from the official `words.txt` annotation file. Lines flagged with an erroneous segmentation (`err`) and images that are missing or corrupt are discarded during loading. The data is split into disjoint training (80%), validation (10%), and test (10%) sets:

$$\mathcal{D} = \mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{val}} \cup \mathcal{D}_{\text{test}}, \quad \mathcal{D}_i \cap \mathcal{D}_j = \emptyset. \tag{1}$$

3.1 Dataset Analysis

Figure 5 shows that the corpus is dominated by short words: one- and two-character tokens are by far the most frequent, while words of more than ten characters are rare. As discussed in Section 5, this length distribution directly shapes the error profile of the model, since longer words give CTC more opportunities to make a per-character mistake.

Historical project report: scores and figures not independently reproduced. See RESULTS.md.

3.2 Image Preprocessing

Every image is processed by a fixed pipeline before entering the network:

1. **Grayscale** conversion to a single channel.
2. **Resize to a fixed height** of H=32 pixels while **preserving the aspect ratio**.
3. **Right-padding** with white pixels to a fixed width W=256, so that all samples share the same tensor shape.
4. **Normalisation** of pixel intensities to $[-1, 1]$ via $x' = (x/255 - 0.5)/0.5$.

Keeping the aspect ratio prevents distortion of the glyph shapes, while fixed-width padding allows efficient mini-batching.

3.3 Character Vocabulary and Label Encoding

We build the character set $\mathcal{V}$ from all characters appearing in the training transcriptions. Index 0 is reserved for the CTC *blank* token, and every character is mapped to an index in $\{1, \ldots, |\mathcal{V}|\}$, giving $C = |\mathcal{V}| + 1$ output classes. A transcription is encoded as the sequence of its character indices; labels of different lengths are concatenated into a single tensor together with a vector of per-sample lengths, which is the input format required by CTC.

4 Methodology

4.1 CRNN Architecture

The model is a CRNN composed of three stages, illustrated in Figures 1 and 2.

Convolutional encoder. A stack of seven convolutional layers (Conv–ReLU with BatchNorm and max-pooling) maps the input image $I \in \mathbb{R}^{1\times 32\times 256}$ to a feature map. Asymmetric pooling collapses the height dimension to 1 while preserving the horizontal resolution, turning the image into a sequence of feature columns:

$$f = \mathrm{CNN}(I) \in \mathbb{R}^{T\times D}, \tag{2}$$

where T is the number of time steps (feature columns) and D=512 the feature dimension.

Bidirectional recurrent encoder. The feature sequence is processed by a two-layer BiLSTM with hidden size 256 per direction. By reading the sequence forwards and backwards, each position is conditioned on both its left and right context, which is essential for disambiguating visually similar glyphs:

$$h_t = [\overrightarrow{h}_t\,;\,\overleftarrow{h}_t] \in \mathbb{R}^{512}. \tag{3}$$

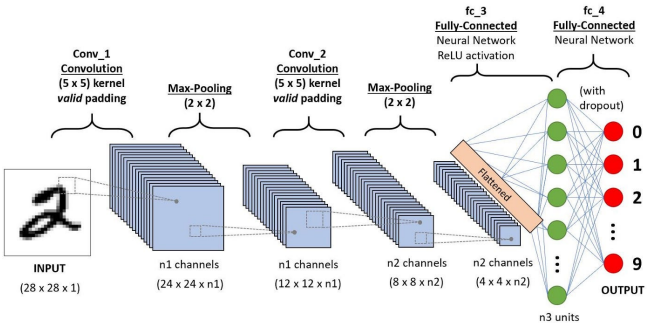


Figure 1: Convolutional encoder. Seven Conv–BatchNorm–ReLU blocks with max-pooling progressively reduce the spatial resolution and collapse the image height to a single row, producing a horizontal sequence of 512-dimensional feature columns.

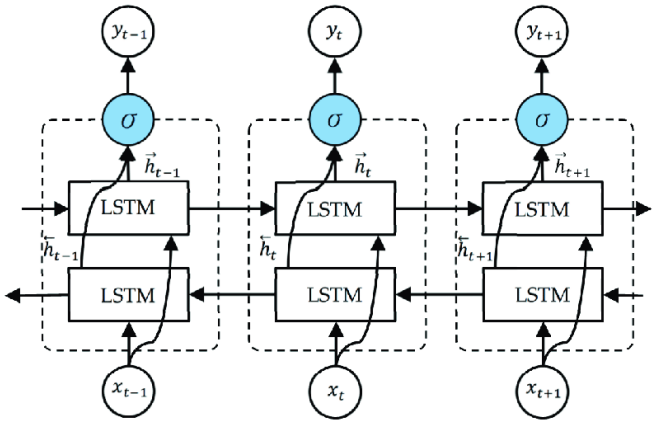


Figure 2: Two-layer bidirectional LSTM sequence model. Each feature column is processed in both directions; the concatenated hidden states are projected by a linear layer to per-step character probabilities.

Linear classifier. A linear layer projects each h_t to class scores, followed by a softmax over the C classes (characters + blank):

$$y_t = \mathrm{softmax}(W h_t + b) \in \mathbb{R}^C. \tag{4}$$

The full output is a matrix $Y \in \mathbb{R}^{T\times C}$.

4.2 CTC Loss

Because the alignment between feature columns and target characters is unknown, we train with the Connectionist Temporal Classification loss. CTC defines a many-to-one mapping $\mathcal{B}$ that removes repeated symbols and blanks (e.g. $\mathcal{B}$(a-aab-) = aab). The probability of a target label ℓ is the sum over all alignments π that map to it:

$$p(\ell \mid I) = \sum_{\pi \in \mathcal{B}^{-1}(\ell)} \prod_{t=1}^{T} y^t_{\pi_t}, \tag{5}$$

Historical project report: scores and figures not independently reproduced. See RESULTS.md.

Table 1: Training hyperparameters.

Hyperparameter	Value
Architecture	CRNN (CNN + 2×BiLSTM)
Input size	32×256 (grayscale)
BiLSTM hidden size	256 per direction
Output classes C	$\lvert\mathcal{V}\rvert + 1$ (blank)
Loss	CTC
Optimizer	Adam
Learning rate	1×10^{-3}
Scheduler	ReduceLROnPlateau
Gradient clip	5.0
Batch size	32
Epochs	100
Decoding	Greedy (best path)
Train/Val/Test	80%/10%/10%

and the loss is the negative log-likelihood:

$$\mathcal{L}_{\text{CTC}} = -\ln p(\ell \mid I). \tag{6}$$

CTC requires the per-sample input lengths (here always T) and the target lengths, which lets it train on variable-length labels without explicit alignment.

4.3 Greedy Decoding

At inference we use greedy (best-path) decoding: at each time step we take the most probable class and then apply $\mathcal{B}$, collapsing repeats and removing blanks:

$$\hat{\ell} = \mathcal{B}\Big(\arg\max_{c} y_c^t \;\Big|\; t = 1, \ldots, T\Big). \tag{7}$$

We additionally report the mean per-step softmax probability of the chosen path as a scalar *confidence* score.

4.4 Training Procedure

The network is optimised end-to-end with **Adam**, gradient clipping ($\|g\| \leq 5$) for stability, and a `ReduceLROnPlateau` schedule driven by the validation loss. The checkpoint with the lowest validation CER is retained. Hyperparameters are listed in Table 1.

4.5 Evaluation Metrics

We report the **Character Error Rate** and **Word Error Rate**, both based on the Levenshtein edit distance:

$$\text{CER} = \frac{S_c + D_c + I_c}{N_c}, \qquad \text{WER} = \frac{S_w + D_w + I_w}{N_w}, \tag{8}$$

where S, D, I are the substitution, deletion, and insertion counts at the character (c) or word (w) level, and N is the number of reference units. Lower is better for both.

Table 2: Final performance.

Metric	Score
Validation CER	0.012
Validation WER	0.027
Exact word accuracy (test, n=200)	89.5%
Correct / total words	179/200

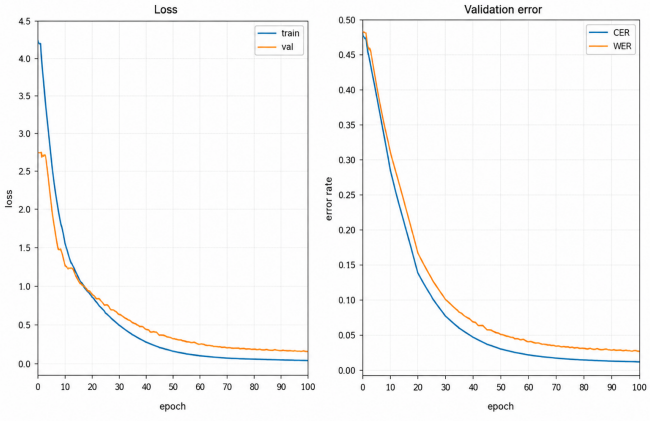


Figure 3: Learning dynamics over 100 epochs. Left: training and validation CTC loss. Right: validation CER and WER. Both error rates converge to well below 0.03.

5 Results

5.1 Quantitative Evaluation

Table 2 reports the final metrics. The model reaches a validation CER of 0.012 and WER of 0.027, and recognises 89.5% of a 200-word held-out test subset exactly (Figure 4).

5.2 Training Dynamics

Figure 3 shows the learning curves over 100 epochs. The training loss decreases monotonically from ≈ 4.2 to ≈ 0.05 and the validation loss from ≈ 2.75 to ≈ 0.15, with only a small final gap and no sign of divergence. The validation CER and WER (right panel) fall sharply from ≈ 0.48 to 0.012 and 0.027 respectively, with the steepest improvement in the first 30 epochs before the schedule reduces the learning rate and the curves flatten.

5.3 Effect of Word Length

Figure 5 relates accuracy to word length. Accuracy is highest for short words (≈ 0.97 for one-to-three characters) and declines smoothly with length, dropping to ≈ 0.31 for words of fifteen or more characters. This is the expected behaviour of a character-level CTC model: exact-match accuracy is roughly the product of the per-character accuracies, so it decays as words grow longer.

Historical project report: scores and figures not independently reproduced. See RESULTS.md.

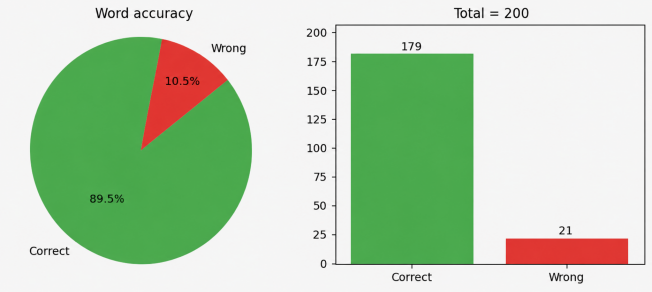


Figure 4: Exact word-match accuracy on a random 200-word test subset: 179 words (89.5%) are recognised correctly and 21 (10.5%) are wrong.

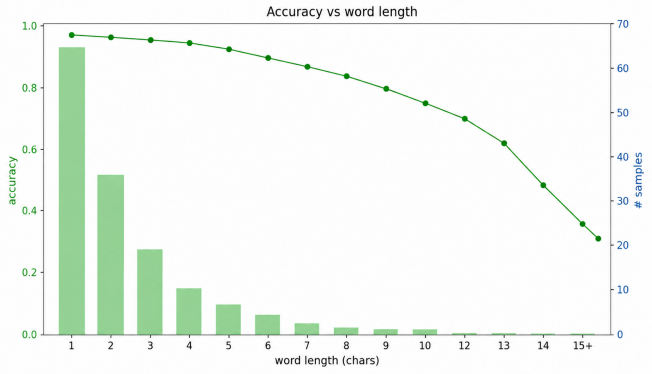


Figure 5: Accuracy versus word length (green line) overlaid on the number of test samples per length (bars). Accuracy degrades on longer words, which are also the rarest.

The bars also confirm that long words are scarce in the dataset, so their lower accuracy has limited impact on the aggregate CER.

5.4 Qualitative Analysis

Figure 6 contrasts the highest- and lowest-confidence predictions. The best cases (*Gaitskellism*, *desolation*, *remained*, *message*, *head-on*) are clearly written and decoded correctly with confidence ≥ 0.95. The worst cases (*correct* $\rightarrow$ *a*, *northern* $\rightarrow$ *a*, *layers* $\rightarrow$ *po*, *control* $\rightarrow$ *wa*) all involve heavy cursive ligatures or unusual slant, and are accompanied by low confidence (0.28–0.34). The strong correlation between low confidence and incorrect output indicates that the model's uncertainty is a useful signal for flagging unreliable predictions. Figure 7 shows a broader sample of correctly recognised words spanning short tokens, punctuation, and hyphenated forms.

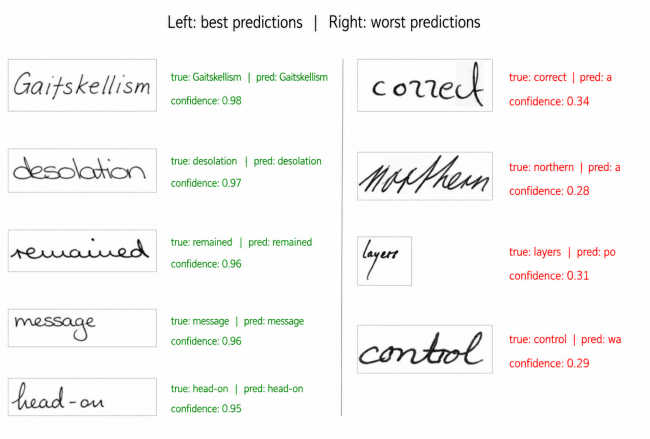


Figure 6: Best (left) and worst (right) predictions ranked by confidence. Green = correct, red = incorrect. High-confidence words are read perfectly; failures occur on heavily cursive inputs and carry low confidence.

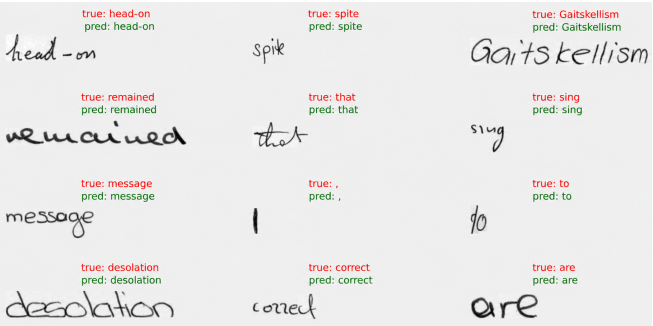


Figure 7: Representative correct predictions on the test set, including short words, punctuation, and hyphenated tokens.

converts the image into a feature sequence, a bidirectional LSTM models the contextual dependencies between feature columns, and a CTC objective trains the whole pipeline end-to-end without any character-level alignment. On the IAM benchmark the model achieves a validation CER of 0.012 and WER of 0.027, and recognises 89.5% of test words exactly. The error analysis shows that mistakes are concentrated on long and heavily cursive words and are reliably accompanied by low model confidence.

Limitations. The system is restricted to single words and cannot directly transcribe full lines or paragraphs. It is trained and evaluated on a single English dataset, so generalisation to other languages, scripts, or acquisition conditions is untested. Greedy decoding uses no lexicon or language model, which limits accuracy on ambiguous cursive inputs.

6 Conclusion

We presented a compact CRNN model for word-level handwritten text recognition. A convolutional encoder

Historical project report: scores and figures not independently reproduced. See RESULTS.md.

7 Future Work

Several extensions would improve the system. **Beam-search decoding** combined with a character- or word-level language model would correct many of the low-confidence errors observed in Figure 6. **Data augmentation** (random affine transforms, slant, and elastic distortion) would improve robustness to cursive and slanted writing. Moving from words to **full text lines** would broaden the scope of the system. Finally, replacing the recurrent encoder with a **Transformer** (e.g. TrOCR) would likely raise accuracy at the cost of additional data and compute.

References

[1] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, "Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks," *ICML*, 2006.

[2] B. Shi, X. Bai, and C. Yao, "An End-to-End Trainable Neural Network for Image-Based Sequence Recognition and Its Application to Scene Text Recognition," *IEEE TPAMI*, 2017.

[3] U.-V. Marti and H. Bunke, "The IAM-database: An English Sentence Database for Offline Handwriting Recognition," *IJDAR*, 2002.

[4] M. Li et al., "TrOCR: Transformer-based Optical Character Recognition with Pre-trained Models," *AAAI*, 2023.